Ultimate DBMS Practical Exam Cheatsheet

1. DATABASE COMMANDS

Create Database

```
CREATE DATABASE demo1;
```

Use Database

```
USE demo1;
```

Show Databases

```
SHOW DATABASES;
```

Show Current Database

```
SELECT DATABASE();
```

Delete Database

```
DROP DATABASE demo1;
```

---

2. TABLE COMMANDS

Create Table

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT
);
```

Show Tables

```
SHOW TABLES;
```

Describe Table

```
DESC Student;
```

Show Create Table

```
SHOW CREATE TABLE Student;
```

Delete Table

```
DROP TABLE Student;
```

Delete Only Data

```
DELETE FROM Student;
```

Delete Data Fast

```
TRUNCATE TABLE Student;
```

---

3. INSERT QUERIES

Insert Single Row

```
INSERT INTO Student
VALUES (1, 'Niraj', 20);
```

Insert Multiple Rows

```
INSERT INTO Student VALUES
(1, 'Niraj', 20),
(2, 'Rahul', 22),
(3, 'Sneha', 19);
```

Safer Insert Syntax

```
INSERT INTO Student
(StudentID, Name, Age)
VALUES
(1, 'Niraj', 20);
```

---

4. SELECT QUERIES

Display All Data

```
SELECT * FROM Student;
```

Display Specific Columns

```
SELECT Name, Age
FROM Student;
```

DISTINCT Values

```
SELECT DISTINCT Address
FROM Student;
```

---

5. WHERE CLAUSE

Greater Than

```
SELECT *
FROM Student
WHERE Age > 20;
```

Less Than

```
SELECT *
FROM Student
WHERE Age < 22;
```

Equal To

```
SELECT *
FROM Student
WHERE Address = 'Pune';
```

AND Operator

```
SELECT *
FROM Student
WHERE Age > 20
AND Address = 'Pune';
```

OR Operator

```
SELECT *
FROM Student
WHERE Address = 'Pune'
OR Address = 'Mumbai';
```

NOT Operator

```sql
SELECT *
FROM Student
WHERE NOT Address = 'Pune';
```

---

6. ORDER BY

Ascending

```sql
SELECT *
FROM Student
ORDER BY Age ASC;
```

Descending

```sql
SELECT *
FROM Student
ORDER BY Age DESC;
```

---

7. LIKE OPERATOR

Starts With

```sql
SELECT *
FROM Student
WHERE Name LIKE 'R%';
```

Ends With

```sql
SELECT *
FROM Student
WHERE Name LIKE '%l';
```

Contains

```
SELECT *
FROM Student
WHERE Name LIKE '%a%';
```

---

8. BETWEEN OPERATOR

```
SELECT *
FROM Student
WHERE Age BETWEEN 18 AND 22;
```

---

9. AGGREGATE FUNCTIONS

COUNT

```
SELECT COUNT(*)
FROM Student;
```

AVG

```
SELECT AVG(Age)
FROM Student;
```

SUM

```
SELECT SUM(Age)
FROM Student;
```

MAX

```
SELECT MAX(Age)
FROM Student;
```

MIN

```sql
SELECT MIN(Age)
FROM Student;
```

---

10. GROUP BY

Count Students Per Course

```sql
SELECT CourseID, COUNT(*)
FROM Enrollment
GROUP BY CourseID;
```

Average Age By Address

```sql
SELECT Address, AVG(Age)
FROM Student
GROUP BY Address;
```

---

11. HAVING CLAUSE

```sql
SELECT CourseID, COUNT(*)
FROM Enrollment
GROUP BY CourseID
HAVING COUNT(*) > 1;
```

---

12. INNER JOIN

Basic Pattern

```sql
SELECT a.col, b.col
FROM table1 a
```

```
INNER JOIN table2 b
ON a.id = b.id;
```

Student + Course

```
SELECT s.Name, c.CourseName
FROM Student s
INNER JOIN Enrollment e
ON s.StudentID = e.StudentID
INNER JOIN Course c
ON e.CourseID = c.CourseID;
```

---

13. LEFT JOIN

```
SELECT s.Name, e.CourseID
FROM Student s
LEFT JOIN Enrollment e
ON s.StudentID = e.StudentID;
```

---

14. RIGHT JOIN

```
SELECT s.Name, e.CourseID
FROM Student s
RIGHT JOIN Enrollment e
ON s.StudentID = e.StudentID;
```

---

15. FULL OUTER JOIN

```
SELECT s.Name, e.CourseID
FROM Student s
LEFT JOIN Enrollment e
ON s.StudentID = e.StudentID

UNION
```

```sql
SELECT s.Name, e.CourseID
FROM Student s
RIGHT JOIN Enrollment e
ON s.StudentID = e.StudentID;
```

---

16. SUBQUERIES

Above Average Age

```sql
SELECT Name
FROM Student
WHERE Age > (
    SELECT AVG(Age)
    FROM Student
);
```

IN Subquery

```sql
SELECT Name
FROM Student
WHERE StudentID IN (
    SELECT StudentID
    FROM Enrollment
);
```

---

17. VIEW

Create View

```sql
CREATE VIEW StudentView AS
SELECT Name, Email
FROM Student;
```

Display View

```sql
SELECT * FROM StudentView;
```

Update View

```sql
UPDATE StudentView
SET Name = 'Rohan'
WHERE Name = 'Rahul';
```

Delete View

```sql
DROP VIEW StudentView;
```

---

18. INDEX

Create Index

```sql
CREATE INDEX idx_email
ON Student(Email);
```

Show Index

```sql
SHOW INDEX FROM Student;
```

---

19. ALTER TABLE

Add Column

```sql
ALTER TABLE Student
ADD Phone VARCHAR(15);
```

Modify Column

```sql
ALTER TABLE Student
MODIFY Name VARCHAR(100);
```

Drop Column

```
ALTER TABLE Student
DROP COLUMN Phone;
```

---

20. UPDATE QUERY

```
UPDATE Student
SET Age = 21
WHERE StudentID = 1;
```

---

21. DELETE QUERY

```
DELETE FROM Student
WHERE StudentID = 1;
```

---

22. PROCEDURES

```
DELIMITER //

CREATE PROCEDURE ShowStudents()
BEGIN
    SELECT * FROM Student;
END //

DELIMITER ;
```

Execute Procedure

```
CALL ShowStudents();
```

---

23. FUNCTIONS

```
DELIMITER //

CREATE FUNCTION StudentCount()
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;

    SELECT COUNT(*)
    INTO total
    FROM Student;

    RETURN total;
END //

DELIMITER ;
```

Execute Function

```
SELECT StudentCount();
```

---

24. TRIGGERS

```
DELIMITER //

CREATE TRIGGER student_trigger
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
    SET NEW.Name = UPPER(NEW.Name);
END //

DELIMITER ;
```

---

25. CURSOR

```
DELIMITER //

CREATE PROCEDURE CursorDemo()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE sname VARCHAR(50);

    DECLARE cur CURSOR FOR
    SELECT Name FROM Student;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET done = TRUE;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO sname;

        IF done THEN
            LEAVE read_loop;
        END IF;

        SELECT sname;
    END LOOP;

    CLOSE cur;
END //

DELIMITER ;
```

---

26. TRANSACTIONS

Start Transaction

```
START TRANSACTION;
```

Commit

```
COMMIT;
```

Rollback

```
ROLLBACK;
```

---

27. BACKUP & RECOVERY

Backup Database

```
mysqldump -u root -p demo1 > backup.sql
```

Restore Database

```
mysql -u root -p demo1 < backup.sql
```

---

28. MONGODB COMMANDS

Create Database

```
use college
```

Insert One

```
db.students.insertOne({
  name: 'Niraj',
  age: 20
})
```

Insert Many

```
db.students.insertMany([
  {name:'Rahul', age:22},
  {name:'Sneha', age:19}
])
```

Find Data

```
db.students.find()
```

Update Data

```
db.students.updateOne(
  {name:'Niraj'},
  {$set:{age:21}}
)
```

Delete Data

```
db.students.deleteOne({name:'Rahul'})
```

---

29. MOST IMPORTANT QUERY PATTERNS

SELECT Pattern

```
SELECT column_name
FROM table_name;
```

WHERE Pattern

```
SELECT column_name
FROM table_name
WHERE condition;
```

GROUP BY Pattern

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name;
```

JOIN Pattern

```
SELECT a.col, b.col
FROM table1 a
JOIN table2 b
ON a.id = b.id;
```

SUBQUERY Pattern

```
SELECT col
FROM table
WHERE value > (
    SELECT AVG(value)
    FROM table
);
```

---

30. MOST IMPORTANT VIVA QUESTIONS

What is Primary Key?

Uniquely identifies each record.

What is Foreign Key?

Links two tables.

What is Join?

Combines data from multiple tables.

What is View?

Virtual table based on query.

What is Index?

Improves search performance.

WHERE vs HAVING?

WHERE filters rows. HAVING filters groups.

What is Subquery?

Query inside another query.

What is Normalization?

Reducing redundancy.

---

31. DEBUGGING COMMANDS

Check Tables

```
SHOW TABLES;
```

Check Database

```
SELECT DATABASE();
```

Check Structure

```
DESC Student;
```

Show Full Create Query

```
SHOW CREATE TABLE Student;
```

---

32. MOST COMMON ERRORS

Foreign Key Error

Parent row missing.

Duplicate Primary Key

Primary key repeated.

Table Doesn't Exist

Wrong table name or database.

Safe Update Error

```
SET SQL_SAFE_UPDATES = 0;
```

---

33. SQL EXECUTION ORDER

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
```

---

34. JOIN EXECUTION MEMORY TRICK

```
SELECT
FROM
JOIN
ON
```

---

35. IMPORTANT MYSQL WORKBENCH SHORTCUTS

Shortcut	Work
Ctrl + Enter	Run current query
Ctrl + Shift + Enter	Run all queries
Ctrl + B	Beautify SQL
Ctrl + /	Comment line
Ctrl + Space	Autocomplete
Ctrl + T	New query tab
Ctrl + F	Find
Ctrl + S	Save

---

36. EXAM STRATEGY

1. CREATE TABLE
2. INSERT DATA
3. SELECT DATA
4. EXECUTE REQUIRED QUERY
5. VERIFY OUTPUT

---

37. GOLDEN RULES

- Parent table first
- Child table later
- Use aliases in joins
- Never forget WHERE in UPDATE/DELETE
- Verify using SELECT *
- Use DESC when confused
- Schema may change but SQL logic stays same